

Ex. No: 6 Semaphore Implementation

PROGRAM 1: MUTUAL EXCLUSION USING SEMAPHORE

C PROGRAM

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <semaphore.h>

#include <sys/mman.h>

#include <sys/wait.h>


int main()


sem_t *sem;


sem = mmap(NULL, sizeof(sem_t),
PROT_READ | PROT_WRITE,
MAP_SHARED | MAP_ANONYMOUS,
-1,0);


sem_init(sem, 1, 1);


if(fork() == 0) sem_wait(sem);


printf("Child Process Entering Critical Section\n");
sleep(3);
printf("Child Process Leaving Critical Section\n");


sem_post(sem);
exit(0);


sem_wait(sem);
```

```
printf("Parent Process Entering Critical Section\n");  
sleep(3);  
printf("Parent Process Leaving Critical Section\n");
```

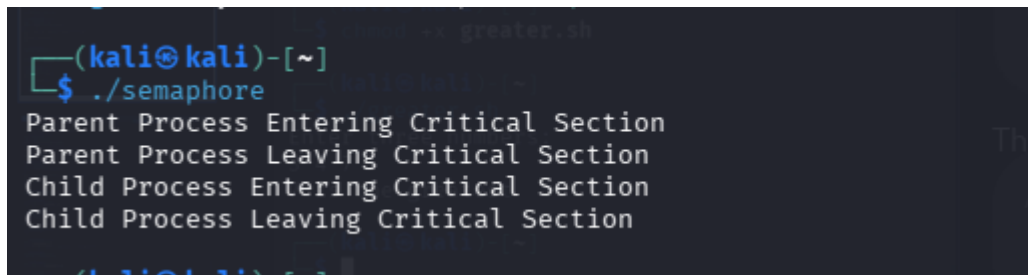
```
sem_post(sem);
```

```
wait(NULL);
```

```
sem_destroy(sem);
```

```
return 0;
```

Output:

A terminal window with a dark background and light blue text. The prompt is (kali㉿kali)-[~]. The user enters ./semaphore. The output shows four lines: Parent Process Entering Critical Section, Parent Process Leaving Critical Section, Child Process Entering Critical Section, and Child Process Leaving Critical Section.

```
(kali㉿kali)-[~]  
$ ./semaphore  
Parent Process Entering Critical Section  
Parent Process Leaving Critical Section  
Child Process Entering Critical Section  
Child Process Leaving Critical Section
```

SHELL SCRIPT:

```
#!/bin/bash  
LOCKFILE="/tmp/mylock"  
while [ -f "$LOCKFILE" ]  
do  
sleep 1  
done  
touch "$LOCKFILE"  
echo "Entering Critical Section"  
sleep 5  
echo "Leaving Critical Section"  
rm -f "$LOCKFILE"
```

Output:

```

➔ chmod +x semaphore.sh
(kali㉿kali)-[~]
$ ./semaphore.sh
Entering Critical Section
Leaving Critical Section

```